

Below is a **complete, structured system and business specification** for the approach you selected:

Core approach:

Verified channel + cryptographic proof + browser extension badge

(with challenge/poll as a secondary layer)

This is written as a **professional system document** that a dev team, investors, or partners can use.

antiai.me

System & Business Specification

Version: 1.0

Type: SaaS Trust & Authenticity Platform

Core Mission:

Provide a simple, scalable, and verifiable way for creators to prove the authenticity of their videos and protect their audience from deepfake impersonation.

1. Executive Summary

1.1 Problem

Deepfake technology enables attackers to:

- Impersonate creators
- Steal audiences
- Spread misinformation
- Damage reputations
- Conduct financial scams

Current solutions:

- Are reactive (platform moderation)
 - Depend on AI detection (unreliable)
 - Do not give creators direct control
-

1.2 Solution

antiai.me provides:

Core system

1. Creator verifies their channel.
2. Each video gets a **cryptographic authenticity proof**.
3. Viewers see a **verification badge** via:
 - Browser extension
 - Verification link
 - Public creator page

Optional second layer

- Object + phrase challenge system
 - Time-bound viewer verification
-

1.3 Key Value Proposition

For creators:

- “Prove your videos are real.”

For viewers:

- “Know instantly if a video is authentic.”

For platforms:

- “Reduce impersonation and fraud risk.”
-

2. System Overview

2.1 Core Components

The system has three major components:

1) Creator Trust System

- Channel verification

- Video registration
- Proof generation

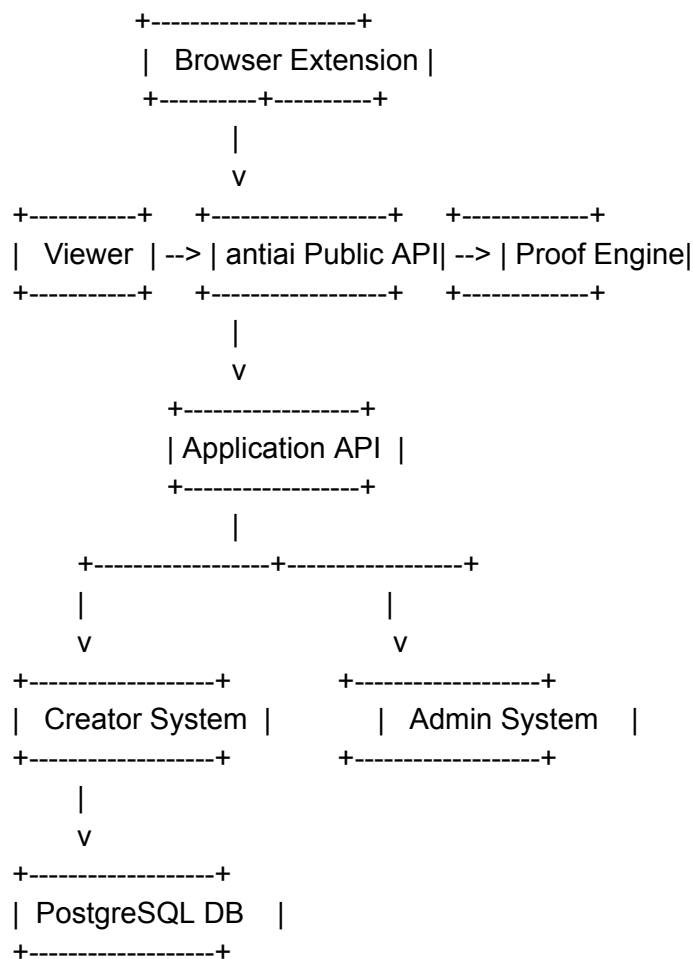
2) Cryptographic Proof Engine

- Signed authenticity tokens
- Public verification endpoints
- Transparency logs

3) Viewer Verification Layer

- Browser extension
- Public verification pages
- API for integrations

3. High-Level Architecture



4. Core Functional Modules

4.1 Creator Account System

Functions:

- Signup/login
- Password or Google OAuth
- Account management
- Subscription billing

4.2 Channel Verification Module

Creators must prove they own the channel.

Supported methods

1. Google OAuth channel ownership
2. Verification token in:
 - Channel description
 - Short verification video
 - Pinned comment

Output

Channel status:

- Pending
- Verified
- Revoked

4.3 Video Registration Module

Creator registers a video.

Flow

1. Enter video URL or select from API.
 2. System fetches:
 - Video ID
 - Channel ID
 - Title
 - Publish time
 3. Creator confirms registration.
-

4.4 Cryptographic Proof Engine

Purpose

Provide a **tamper-proof authenticity signal**.

How it works

For each registered video:

System generates:

```
proof_payload = {  
  video_id,  
  channel_id,  
  creator_id,  
  timestamp,  
  expiry,  
  nonce  
}
```

Then:

```
proof_signature = Sign(payload, platform_private_key)
```

Final proof token:

```
base64(payload + signature)
```

Storage

Proof stored in:

- Database
 - Public verification endpoint
 - Transparency log
-

Verification

Extension or API:

1. Fetch proof.
 2. Validate signature.
 3. Check expiry.
 4. Compare video ID.
-

4.5 Transparency Log

Public record of:

- Channel verifications
- Video proofs
- Revocations

Purpose:

- Public trust
 - Legal evidence
 - Audit trail
-

4.6 Browser Extension (Core User-Facing Feature)

Function

When a user watches a video:

1. Extension detects video ID.
2. Calls:

GET /public/verify?video_id=XXXX

3. Receives:
 - Verified

- Unverified
 - Suspicious
-

Badge states

State	Meaning
 Verified	Authentic creator-verified video
 Unverified	No authenticity proof
 Suspicious	Known impersonation or mismatch

4.7 Challenge System (Optional Layer)

Secondary defense.

Flow

1. Creator creates challenge.
2. System assigns:
 - Random object
 - Random phrase
3. Creator shows them in video.
4. Viewers answer poll.

Poll:

- Time-bound
 - Session-limited
 - Rotating decoys
-

5. Database Design

5.1 Core Tables

users

- id
 - email
 - password_hash
 - role
 - created_at
-

channels

- id
 - user_id
 - youtube_channel_id
 - channel_name
 - avatar_url
 - verification_status
 - verified_at
-

videos

- id
 - channel_id
 - youtube_video_id
 - title
 - published_at
 - created_at
-

proofs

- id
 - video_id
 - payload_json
 - signature
 - created_at
 - expires_at
 - status
-

challenges

- id
 - video_id
 - object_code
 - phrase
 - starts_at
 - expires_at
 - status
-

attempts

- id
 - challenge_id
 - selected_option
 - is_correct
 - ip_hash
 - created_at
-

subscriptions

- id
 - user_id
 - stripe_customer_id
 - plan
 - status
 - current_period_end
-

audit_logs

- id
 - user_id
 - action
 - entity
 - metadata
 - created_at
-

6. API Design

6.1 Public API

Used by extension and third parties.

Verify video

GET /public/verify?video_id=XXXX

Response:

```
{  
  "status": "verified",  
  "channel_name": "...",  
  "verified_at": "...",  
  "expires_at": "..."  
}
```

6.2 Creator APIs

Create proof

POST /videos/{id}/proof

List proofs

GET /proofs

6.3 Challenge APIs

POST /challenges

GET /challenges/{id}

POST /challenges/{id}/attempt

6.4 Billing APIs

POST /billing/checkout

POST /billing/webhook

GET /billing/status

7. Subscription Model

7.1 Plans

Free

- 5 verified videos/month
- Basic badge
- No analytics

Pro (\$9–\$19/month)

- 100 verified videos/month
- Analytics dashboard
- Challenge system

Elite (\$39–\$79/month)

- Unlimited videos
 - Team members
 - API access
 - Advanced fraud detection
-

7.2 Billing logic

- Stripe subscription
 - Webhook updates status
 - Backend enforces limits
-

8. Security Model

8.1 Core principles

- Never trust client-side data.

- All proofs signed server-side.
 - All verification stateless.
-

8.2 Protection measures

- Rate limiting per IP.
 - Bot detection.
 - Signed proof tokens.
 - Expiry enforcement.
 - Audit logs.
-

9. Browser Extension Design

9.1 Core behavior

On YouTube video page:

1. Extract video ID.
 2. Query API.
 3. Render badge overlay.
-

9.2 Minimal MVP extension

Features:

- Video authenticity badge
 - Click to see details
 - Link to creator page
-

10. MVP Scope (First Launch)

Must-have

- Creator signup
 - Channel verification
 - Video proof creation
 - Public verify endpoint
 - Basic browser extension
 - Free + Pro plan
-

Not in MVP

- Full challenge system
 - AI detection
 - Mobile apps
 - Team accounts
-

11. Phase 2 Roadmap

Phase 2

- Object + phrase challenges
- Team accounts
- Public creator pages
- Advanced analytics

Phase 3

- Platform partnerships
 - C2PA-style signing
 - Mobile verification apps
 - AI-based impersonation alerts
-

12. Tech Stack Recommendation

Frontend

- Next.js (App Router)
- Tailwind CSS

Backend

- Node.js (NestJS) or Python (FastAPI)

Database

- PostgreSQL

Cache

- Redis

Payments

- Stripe

Hosting

- Vercel (frontend)
 - AWS/GCP (backend + DB)
-

13. Key Success Metrics

Creator metrics

- Verified channels
- Videos registered per creator
- Subscription conversion

Viewer metrics

- Extension installs
 - Daily active extension users
 - Verification checks per day
-

14. Go-to-Market Strategy

Phase 1: Niche creators

Target:

- Crypto YouTubers
 - Finance channels
 - Political commentators
 - Educators
-

Phase 2: Partnerships

- News outlets
 - Influencer agencies
 - Podcast networks
-

15. Biggest Risks

Risk		Mitigation	
Low	viewer	Browser extension	
adoption			
Creator friction		One-click generation	proof
Platform dependency		Independent verification layer	
Fast	deepfake	Dual-layer system	
attackers			

Final Business Positioning

antiai.me should be marketed as:

Not:

“Deepfake prevention system”

But as:

“The authenticity badge for real creators.”

```
-- antiai.me (PostgreSQL 14+)
-- Final schema: Ed25519-only + re-issuable proofs with strict rules (versioned, immutable, one
ACTIVE per video)
-- Key properties:
-- - Proofs are append-only: never update payload/signature after creation.
-- - Reissue creates a new proof row and supersedes the prior active proof.
-- - Only one ACTIVE proof per video enforced by partial unique index.
-- - Ed25519-only enforced by CHECK constraints.
-- - Public keys are served via signing_keys table (rotation-ready).
```

BEGIN;

```
-- Extensions
```

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

```
CREATE EXTENSION IF NOT EXISTS citext;
```

```
-- ----- ENUMS -----
```

```
DO $$ BEGIN
```

```
    CREATE TYPE user_role AS ENUM ('creator','admin');
```

```
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
```

```
DO $$ BEGIN
```

```
    CREATE TYPE verification_status AS ENUM ('pending','verified','revoked');
```

```
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
```

```
-- Proof status includes 'superseded' for normal re-issues (cleaner than "revoked")
```

```
DO $$ BEGIN
```

```
    CREATE TYPE proof_status AS ENUM ('active','expired','revoked','superseded');
```

```
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
```

```
DO $$ BEGIN
```

```
    CREATE TYPE challenge_status AS ENUM ('active','paused','expired');
```

```
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
```

```
DO $$ BEGIN
```

```
    CREATE TYPE challenge_type AS ENUM ('object','phrase','object_phrase');
```



```
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
```

```
DO $$ BEGIN
```

```
    CREATE TYPE report_status AS ENUM ('open','reviewed','closed');
```

```
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
```

```
DO $$ BEGIN
```

```
    CREATE TYPE subscription_status AS ENUM  
('trialing','active','past_due','canceled','unpaid','incomplete','incomplete_expired');
```

```
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
```

```
DO $$ BEGIN
```

```
    CREATE TYPE plan_tier AS ENUM ('free','pro','elite');
```

```
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
```

```
-- ----- UPDATED_AT TRIGGER -----
```

```
CREATE OR REPLACE FUNCTION set_updated_at()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    NEW.updated_at = NOW();
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
-- ----- USERS -----
```

```
CREATE TABLE IF NOT EXISTS users (
```

```
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    email       CITEXT UNIQUE NOT NULL,
```

```
    password_hash TEXT, -- nullable if OAuth-only
```

```
    role        user_role NOT NULL DEFAULT 'creator',
```

```
    is_email_verified BOOLEAN NOT NULL DEFAULT FALSE,
```

```
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
    last_login_at TIMESTAMPTZ
```

```
);
```

```
DROP TRIGGER IF EXISTS trg_users_updated_at ON users;
```

```
CREATE TRIGGER trg_users_updated_at
```

```
BEFORE UPDATE ON users
```

```
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

```
-- ----- OAUTH ACCOUNTS -----
```

```
CREATE TABLE IF NOT EXISTS oauth_accounts (
```

```
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

user_id          UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
provider         TEXT NOT NULL CHECK (provider IN ('google')),
provider_user_id TEXT NOT NULL,
access_token_enc BYTEA,
refresh_token_enc BYTEA,
token_expiry     TIMESTAMPTZ,
scope           TEXT,
created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE (provider, provider_user_id),
UNIQUE (user_id, provider)
);

```

```

DROP TRIGGER IF EXISTS trg_oauth_accounts_updated_at ON oauth_accounts;
CREATE TRIGGER trg_oauth_accounts_updated_at
BEFORE UPDATE ON oauth_accounts
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

```

-- ----- SIGNING KEYS (Ed25519 public keys) -----
CREATE TABLE IF NOT EXISTS signing_keys (
  id          TEXT PRIMARY KEY, -- kid, e.g. "k_2026_01"
  alg         TEXT NOT NULL DEFAULT 'Ed25519' CHECK (alg = 'Ed25519'),
  public_key_b64 TEXT NOT NULL, -- base64 of raw 32-byte Ed25519 public key
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  retired_at  TIMESTAMPTZ
);

```

```

CREATE INDEX IF NOT EXISTS idx_signing_keys_active ON signing_keys(is_active);

```

```

-- ----- CHANNELS -----
CREATE TABLE IF NOT EXISTS channels (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  youtube_channel_id TEXT NOT NULL UNIQUE,
  channel_handle TEXT,
  channel_name TEXT NOT NULL,
  channel_url TEXT,
  avatar_url TEXT,
  verification_status verification_status NOT NULL DEFAULT 'pending',
  verification_method TEXT, -- 'oauth' | 'about_token' | 'video_token' | 'pinned_comment'
  verified_at TIMESTAMPTZ,
  revoked_at TIMESTAMPTZ,
  revoke_reason TEXT,

```

```
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (user_id, youtube_channel_id)
);
```

```
DROP TRIGGER IF EXISTS trg_channels_updated_at ON channels;
CREATE TRIGGER trg_channels_updated_at
BEFORE UPDATE ON channels
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

```
CREATE INDEX IF NOT EXISTS idx_channels_user  ON channels(user_id);
CREATE INDEX IF NOT EXISTS idx_channels_status ON channels(verification_status);
```

-- ----- VIDEOS -----

```
CREATE TABLE IF NOT EXISTS videos (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    channel_id   UUID NOT NULL REFERENCES channels(id) ON DELETE CASCADE,
    youtube_video_id TEXT NOT NULL UNIQUE,
    title        TEXT,
    video_url     TEXT,
    published_at  TIMESTAMPTZ,
    registered_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX IF NOT EXISTS idx_videos_channel  ON videos(channel_id);
CREATE INDEX IF NOT EXISTS idx_videos_published ON videos(published_at);
```

-- ----- PROOFS (versioned, re-issuable) -----

-- STRICT RULES:

```
-- - alg must be Ed25519
-- - payload_b64 + signature_b64 are immutable for a given row
-- - multiple proofs per video allowed
-- - only ONE active proof per video enforced via partial unique index
-- - reissue uses supersedes_proof_id + superseded_at and sets old to 'superseded'
```

```
CREATE TABLE IF NOT EXISTS proofs (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    video_id     UUID NOT NULL REFERENCES videos(id) ON DELETE CASCADE,
    channel_id   UUID NOT NULL REFERENCES channels(id) ON DELETE CASCADE,
```

-- Signing material

```
alg          TEXT NOT NULL DEFAULT 'Ed25519' CHECK (alg = 'Ed25519'),
kid          TEXT NOT NULL REFERENCES signing_keys(id) ON DELETE RESTRICT,
```

```

-- Canonical payload & signature
payload_json      JSONB NOT NULL, -- stored for debugging/inspection; canonical bytes
must match payload_b64
payload_b64       TEXT NOT NULL, -- base64url(canonical JSON bytes)
signature_b64     TEXT NOT NULL, -- base64url(Ed25519 signature 64 bytes)

issued_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
expires_at       TIMESTAMPTZ NOT NULL,
status           proof_status NOT NULL DEFAULT 'active',

-- Version lineage for re-issue
supersedes_proof_id  UUID REFERENCES proofs(id) ON DELETE SET NULL,
superseded_at       TIMESTAMPTZ,

-- Revocation (only for security/abuse). Normal re-issue should be status='superseded'
revoked_at        TIMESTAMPTZ,
revoke_reason     TEXT,

created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_proofs_video      ON proofs(video_id);
CREATE INDEX IF NOT EXISTS idx_proofs_channel    ON proofs(channel_id);
CREATE INDEX IF NOT EXISTS idx_proofs_status_exp ON proofs(status, expires_at);
CREATE INDEX IF NOT EXISTS idx_proofs_supersedes ON proofs(supersedes_proof_id);

-- Enforce one ACTIVE proof per video
CREATE UNIQUE INDEX IF NOT EXISTS uq_proofs_video_active
ON proofs(video_id)
WHERE status = 'active';

-- (Optional but recommended) Prevent multiple proofs from claiming to supersede the same
proof
CREATE UNIQUE INDEX IF NOT EXISTS uq_proofs_supersedes_unique
ON proofs(supersedes_proof_id)
WHERE supersedes_proof_id IS NOT NULL;

-- ----- TRANSPARENCY LOG -----
CREATE TABLE IF NOT EXISTS transparency_log (
  id          BIGSERIAL PRIMARY KEY,
          event_type          TEXT          NOT          NULL,      --
'channel_verified','proof_issued','proof_superseded','proof_revoked','channel_revoked'
  entity_type TEXT NOT NULL, -- 'channel','proof'
  entity_id   UUID NOT NULL,

```

```
    event_time TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    data       JSONB NOT NULL
);
```

```
CREATE INDEX IF NOT EXISTS idx_tlog_entity ON transparency_log(entity_type, entity_id);
CREATE INDEX IF NOT EXISTS idx_tlog_time  ON transparency_log(event_time DESC);
```

-- ----- OPTIONAL: CHALLENGES -----

```
CREATE TABLE IF NOT EXISTS challenges (
    id          TEXT PRIMARY KEY, -- short id for URL
    video_id    UUID NOT NULL REFERENCES videos(id) ON DELETE CASCADE,
    channel_id  UUID NOT NULL REFERENCES channels(id) ON DELETE CASCADE,
    type        challenge_type NOT NULL DEFAULT 'object',
    object_code TEXT,
    object_label TEXT,
    phrase      TEXT,
    starts_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expires_at  TIMESTAMPTZ NOT NULL,
    status      challenge_status NOT NULL DEFAULT 'active',
    decoy_pool_version INT NOT NULL DEFAULT 1,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX IF NOT EXISTS idx_challenges_video  ON challenges(video_id);
CREATE INDEX IF NOT EXISTS idx_challenges_channel ON challenges(channel_id);
CREATE INDEX IF NOT EXISTS idx_challenges_status_exp ON challenges(status,
expires_at);
```

```
CREATE TABLE IF NOT EXISTS challenge_options (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    challenge_id TEXT NOT NULL REFERENCES challenges(id) ON DELETE CASCADE,
    option_code  TEXT NOT NULL,
    option_label TEXT NOT NULL,
    is_correct  BOOLEAN NOT NULL DEFAULT FALSE,
    sort_order  INT NOT NULL DEFAULT 0
);
```

```
CREATE INDEX IF NOT EXISTS idx_challenge_options_ch ON
challenge_options(challenge_id);
```

```
CREATE TABLE IF NOT EXISTS challenge_attempts (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    challenge_id TEXT NOT NULL REFERENCES challenges(id) ON DELETE
CASCADE,
```

```

    selected_option_id    UUID REFERENCES challenge_options(id) ON DELETE SET NULL,
    is_correct            BOOLEAN NOT NULL,
    ip_hash               BYTEA,
    user_agent_hash       BYTEA,
    device_fingerprint_hash BYTEA,
    created_at            TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX IF NOT EXISTS idx_attempts_ch_time ON challenge_attempts(challenge_id,
created_at DESC);
CREATE INDEX IF NOT EXISTS idx_attempts_ip_time ON challenge_attempts(ip_hash,
created_at DESC);

```

-- ----- BILLING -----

```

CREATE TABLE IF NOT EXISTS subscriptions (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    plan              plan_tier NOT NULL DEFAULT 'free',
    status             subscription_status NOT NULL DEFAULT 'active',
    stripe_customer_id TEXT,
    stripe_subscription_id TEXT,
    current_period_end TIMESTAMPTZ,
    cancel_at_period_end BOOLEAN NOT NULL DEFAULT FALSE,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (user_id)
);

```

```

DROP TRIGGER IF EXISTS trg_subscriptions_updated_at ON subscriptions;
CREATE TRIGGER trg_subscriptions_updated_at
BEFORE UPDATE ON subscriptions
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

```

CREATE INDEX IF NOT EXISTS idx_subscriptions_status ON subscriptions(status);

```

-- ----- AUDIT LOGS -----

```

CREATE TABLE IF NOT EXISTS audit_logs (
    id                BIGSERIAL PRIMARY KEY,
    user_id           UUID REFERENCES users(id) ON DELETE SET NULL,
    action             TEXT NOT NULL,
    entity_type        TEXT,
    entity_id          TEXT,
    metadata           JSONB NOT NULL DEFAULT '{}::jsonb',
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

);

CREATE INDEX IF NOT EXISTS idx_audit_user_time ON audit_logs(user_id, created_at DESC);

CREATE INDEX IF NOT EXISTS idx_audit_action_time ON audit_logs(action, created_at DESC);

-- ----- REPORTS -----

CREATE TABLE IF NOT EXISTS reports (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 channel_id UUID REFERENCES channels(id) ON DELETE SET NULL,
 video_id UUID REFERENCES videos(id) ON DELETE SET NULL,
 proof_id UUID REFERENCES proofs(id) ON DELETE SET NULL,
 challenge_id TEXT REFERENCES challenges(id) ON DELETE SET NULL,
 reporter_ip_hash BYTEA,
 reason TEXT NOT NULL,
 details TEXT,
 status report_status NOT NULL DEFAULT 'open',
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 reviewed_at TIMESTAMPTZ,
 reviewed_by UUID REFERENCES users(id) ON DELETE SET NULL
);

CREATE INDEX IF NOT EXISTS idx_reports_status_time ON reports(status, created_at DESC);

COMMIT;

openapi: 3.0.3

info:

title: antiai.me API

version: "1.0.0"

description: |

antiai.me API: verified channel + Ed25519 cryptographic proofs + browser extension verification.

Proofs are re-issuable with strict rules:

- Proof rows are immutable once created.
- Re-issue creates a new proof row and marks the prior active proof as "superseded".
- Only one ACTIVE proof per video at a time.

servers:

- url: https://api.antiai.me
description: Production
- url: https://staging-api.antiai.me
description: Staging

tags:

- name: Auth
- name: Channels
- name: Videos
- name: Proofs
- name: Public
- name: Billing
- name: Challenges
- name: Admin

components:

securitySchemes:

BearerAuth:

type: http

scheme: bearer

bearerFormat: JWT

parameters:

YoutubeVideoIdQuery:

name: youtube_video_id

in: query

required: true

schema:

type: string

description: YouTube video ID (e.g., dQw4w9WgXcQ)

ProofIdPath:

name: proof_id

in: path

required: true

schema:

type: string

format: uuid

schemas:

Error:

type: object

required: [error, message]

properties:
 error:
 type: string
 example: bad_request
 message:
 type: string
 example: Invalid input
 details:
 type: object
 additionalProperties: true

AuthTokenResponse:
 type: object
 required: [access_token, token_type, expires_in]
 properties:
 access_token:
 type: string
 description: JWT access token
 token_type:
 type: string
 example: Bearer
 expires_in:
 type: integer
 example: 3600

SignupRequest:
 type: object
 required: [email, password]
 properties:
 email:
 type: string
 format: email
 password:
 type: string
 minLength: 8

LoginRequest:
 type: object
 required: [email, password]
 properties:
 email:
 type: string
 format: email
 password:

type: string

MeResponse:

type: object

required: [id, email, role]

properties:

id:

type: string

format: uuid

email:

type: string

format: email

role:

type: string

enum: [creator, admin]

Channel:

type: object

required: [id, youtube_channel_id, channel_name, verification_status]

properties:

id:

type: string

format: uuid

youtube_channel_id:

type: string

channel_handle:

type: string

nullable: true

channel_name:

type: string

channel_url:

type: string

nullable: true

avatar_url:

type: string

nullable: true

verification_status:

type: string

enum: [pending, verified, revoked]

verified_at:

type: string

format: date-time

nullable: true

ChannelVerifyStartRequest:

type: object
required: [youtube_channel_id, method]
properties:
 youtube_channel_id:
 type: string
 method:
 type: string
 enum: [oauth, about_token, video_token, pinned_comment]
 requested_handle:
 type: string
 nullable: true

ChannelVerifyStartResponse:

type: object
required: [channel_id, method, token_expires_at]
properties:
 channel_id:
 type: string
 format: uuid
 method:
 type: string
 token:
 type: string
 nullable: true
 description: Verification token to place as instructed (null for oauth method)
 instructions:
 type: string
 nullable: true
 token_expires_at:
 type: string
 format: date-time

ChannelVerifyConfirmRequest:

type: object
required: [channel_id]
properties:
 channel_id:
 type: string
 format: uuid

Video:

type: object
required: [id, youtube_video_id, channel_id]

properties:
 id:
 type: string
 format: uuid
 channel_id:
 type: string
 format: uuid
 youtube_video_id:
 type: string
 title:
 type: string
 nullable: true
 video_url:
 type: string
 nullable: true
 published_at:
 type: string
 format: date-time
 nullable: true

VideolImportRequest:

type: object
required: [video_url]
properties:
 video_url:
 type: string
 description: Full YouTube URL
 channel_id:
 type: string
 format: uuid
 nullable: true
 description: Optional override if user has multiple verified channels

SigningKey:

type: object
required: [kid, alg, public_key_b64]
properties:
 kid:
 type: string
 alg:
 type: string
 enum: [Ed25519]
 public_key_b64:
 type: string

description: base64 of raw 32-byte Ed25519 public key

Proof:

type: object

required:

[id, video_id, channel_id, alg, kid, payload_b64, signature_b64, issued_at, expires_at, status]

properties:

id:

type: string

format: uuid

video_id:

type: string

format: uuid

channel_id:

type: string

format: uuid

alg:

type: string

enum: [Ed25519]

kid:

type: string

description: Key id used to sign (for verification/rotation)

payload_json:

type: object

additionalProperties: true

description: Stored for inspection; verification should use payload_b64 bytes.

payload_b64:

type: string

description: base64url(canonical JSON bytes)

signature_b64:

type: string

description: base64url(Ed25519 signature; 64 raw bytes)

issued_at:

type: string

format: date-time

expires_at:

type: string

format: date-time

status:

type: string

enum: [active, expired, revoked, superseded]

supersedes_proof_id:

type: string

format: uuid
nullable: true
superseded_at:
type: string
format: date-time
nullable: true
revoked_at:
type: string
format: date-time
nullable: true
revoke_reason:
type: string
nullable: true

ProofIssueRequest:
type: object
required: [video_id, expires_at]
properties:
video_id:
type: string
format: uuid
expires_at:
type: string
format: date-time

ProofReissueRequest:
type: object
required: [video_id, expires_at, reason]
properties:
video_id:
type: string
format: uuid
expires_at:
type: string
format: date-time
reason:
type: string
enum: [extend_expiry, key_rotation, security_incident]
note:
type: string
nullable: true

ProofReissueResponse:
type: object

required: [new_proof, old_proof]
properties:
 new_proof:
 \$ref: "#/components/schemas/Proof"
 old_proof:
 \$ref: "#/components/schemas/Proof"

PublicVerifyResponse:

type: object
required: [status, youtube_video_id]
properties:
 status:
 type: string
 enum: [verified, unverified, revoked, expired]
 youtube_video_id:
 type: string
 youtube_channel_id:
 type: string
 nullable: true
 channel_name:
 type: string
 nullable: true
 channel_handle:
 type: string
 nullable: true
 proof:
 \$ref: "#/components/schemas/Proof"
 public_creator_url:
 type: string
 nullable: true
 message:
 type: string
 nullable: true

BillingCheckoutRequest:

type: object
required: [plan, success_url, cancel_url]
properties:
 plan:
 type: string
 enum: [pro, elite]
 success_url:
 type: string
 cancel_url:

type: string

BillingCheckoutResponse:

type: object

required: [checkout_url]

properties:

checkout_url:

type: string

BillingStatusResponse:

type: object

required: [plan, status]

properties:

plan:

type: string

enum: [free, pro, elite]

status:

type: string

enum: [trialing, active, past_due, canceled, unpaid, incomplete, incomplete_expired]

current_period_end:

type: string

format: date-time

nullable: true

ChallengeCreateRequest:

type: object

required: [video_id, type, expires_at]

properties:

video_id:

type: string

format: uuid

type:

type: string

enum: [object, phrase, object_phrase]

expires_at:

type: string

format: date-time

allow_reroll:

type: boolean

default: true

ChallengePublic:

type: object

required: [id, type, status, expires_at, video]

properties:
 id:
 type: string
 type:
 type: string
 enum: [object, phrase, object_phrase]
 status:
 type: string
 enum: [active, paused, expired]
 expires_at:
 type: string
 format: date-time
 video:
 type: object
 required: [youtube_video_id, title]
 properties:
 youtube_video_id:
 type: string
 title:
 type: string
 question:
 type: string
 options:
 type: array
 items:
 type: object
 required: [option_id, label]
 properties:
 option_id:
 type: string
 format: uuid
 label:
 type: string

ChallengeAttemptRequest:

type: object
required: [option_id]
properties:
 option_id:
 type: string
 format: uuid

ChallengeAttemptResponse:

type: object

required: [is_correct, status]
properties:
 is_correct:
 type: boolean
 status:
 type: string
 enum: [passed, failed, expired, paused]
 message:
 type: string

paths:

/health:

get:

tags: [Public]

summary: Health check

responses:

"200":

description: OK

/auth/signup:

post:

tags: [Auth]

summary: Create account

requestBody:

required: true

content:

application/json:

schema: { \$ref: "#/components/schemas/SignupRequest" }

responses:

"200":

description: Created

content:

application/json:

schema: { \$ref: "#/components/schemas/AuthTokenResponse" }

"400":

description: Invalid input

content:

application/json:

schema: { \$ref: "#/components/schemas/Error" }

/auth/login:

post:

tags: [Auth]

summary: Login

```
requestBody:
  required: true
  content:
    application/json:
      schema: { $ref: "#/components/schemas/LoginRequest" }
responses:
  "200":
    description: Logged in
    content:
      application/json:
        schema: { $ref: "#/components/schemas/AuthTokenResponse" }
  "401":
    description: Unauthorized
    content:
      application/json:
        schema: { $ref: "#/components/schemas/Error" }
```

```
/auth/me:
  get:
    tags: [Auth]
    summary: Current user
    security: [{ BearerAuth: [] }]
    responses:
      "200":
        description: Me
        content:
          application/json:
            schema: { $ref: "#/components/schemas/MeResponse" }
      "401":
        description: Unauthorized
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Error" }
```

```
/oauth/google/start:
  get:
    tags: [Auth]
    summary: Start Google OAuth for YouTube
    responses:
      "302":
        description: Redirect
```

```
/oauth/google/callback:
  get:
```

tags: [Auth]
summary: Google OAuth callback
responses:
 "302":
 description: Redirect back to app

/channels:
get:
 tags: [Channels]
 summary: List creator channels
 security: [{ BearerAuth: [] }]
 responses:
 "200":
 description: Channels
 content:
 application/json:
 schema:
 type: object
 required: [items]
 properties:
 items:
 type: array
 items: { \$ref: "#/components/schemas/Channel" }

/channels/verify/start:
post:
 tags: [Channels]
 summary: Start channel verification
 security: [{ BearerAuth: [] }]
 requestBody:
 required: true
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/ChannelVerifyStartRequest" }
 responses:
 "200":
 description: Started
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/ChannelVerifyStartResponse" }
 "400":
 description: Invalid request
 content:
 application/json:

schema: { \$ref: "#/components/schemas/Error" }

/channels/verify/confirm:

post:

tags: [Channels]

summary: Confirm channel verification

security: [{ BearerAuth: [] }]

requestBody:

required: true

content:

application/json:

schema: { \$ref: "#/components/schemas/ChannelVerifyConfirmRequest" }

responses:

"200":

description: Verified (or still pending)

content:

application/json:

schema: { \$ref: "#/components/schemas/Channel" }

"409":

description: Not verified yet / token missing

content:

application/json:

schema: { \$ref: "#/components/schemas/Error" }

/videos/import:

post:

tags: [Videos]

summary: Import/register a video by URL

security: [{ BearerAuth: [] }]

requestBody:

required: true

content:

application/json:

schema: { \$ref: "#/components/schemas/VideoImportRequest" }

responses:

"200":

description: Imported

content:

application/json:

schema: { \$ref: "#/components/schemas/Video" }

"400":

description: Invalid URL or not owned by channel

content:

application/json:

schema: { \$ref: "#/components/schemas/Error" }

/videos:

get:

tags: [Videos]

summary: List videos for creator channels

security: [{ BearerAuth: [] }]

parameters:

- name: channel_id

in: query

required: false

schema: { type: string, format: uuid }

responses:

"200":

description: Videos

content:

application/json:

schema:

type: object

required: [items]

properties:

items:

type: array

items: { \$ref: "#/components/schemas/Video" }

/proofs/issue:

post:

tags: [Proofs]

summary: Issue Ed25519 proof for a registered video (creates ACTIVE proof if none active exists)

security: [{ BearerAuth: [] }]

requestBody:

required: true

content:

application/json:

schema: { \$ref: "#/components/schemas/ProofIssueRequest" }

responses:

"200":

description: Proof issued

content:

application/json:

schema: { \$ref: "#/components/schemas/Proof" }

"409":

description: Active proof already exists (use reissue)

```

    content:
      application/json:
        schema: { $ref: "#/components/schemas/Error" }
  "403":
    description: Channel not verified / not owner
    content:
      application/json:
        schema: { $ref: "#/components/schemas/Error" }

/proofs/reissue:
  post:
    tags: [Proofs]
    summary: Re-issue proof (strict rules: creates new ACTIVE proof and supersedes prior ACTIVE)
    security: [{ BearerAuth: [] }]
    description: |
      Creates a new proof row with status=active and sets the prior active proof to status=superseded.
      Allowed reasons: extend_expiry, key_rotation, security_incident.
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: "#/components/schemas/ProofReissueRequest" }
    responses:
      "200":
        description: Reissued
        content:
          application/json:
            schema: { $ref: "#/components/schemas/ProofReissueResponse" }
      "404":
        description: No active proof exists to reissue (use issue)
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Error" }
      "403":
        description: Not owner / channel not verified
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Error" }

/proofs:
  get:
    tags: [Proofs]

```

summary: List proofs for current user (includes superseded history)

security: [{ BearerAuth: [] }]

parameters:

- name: video_id
 - in: query
 - required: false
 - schema: { type: string, format: uuid }

responses:

"200":
description: Proofs
content:
application/json:
schema:
type: object
required: [items]
properties:
items:
type: array
items: { \$ref: "#/components/schemas/Proof" }

/public/verify:

get:

tags: [Public]

summary: Verify a YouTube video (used by browser extension) - returns ACTIVE proof if present

parameters:

- \$ref: "#/components/parameters/YoutubeVideoIdQuery"

responses:

"200":
description: Verification result
content:
application/json:
schema: { \$ref: "#/components/schemas/PublicVerifyResponse" }

/public/proof:

get:

tags: [Public]

summary: Fetch ACTIVE proof token (payload + signature) by YouTube video ID

parameters:

- \$ref: "#/components/parameters/YoutubeVideoIdQuery"

responses:

"200":
description: ACTIVE proof token
content:

application/json:
 schema:
 type: object
 required: [alg, kid, payload_b64, signature_b64, expires_at]
 properties:
 alg:
 type: string
 enum: [Ed25519]
 kid:
 type: string
 payload_b64:
 type: string
 signature_b64:
 type: string
 expires_at:
 type: string
 format: date-time

"404":
 description: No active proof
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/Error" }

/public/proofs/{proof_id}:
 get:
 tags: [Public]
 summary: Fetch a specific proof by id (auditing / transparency)
 parameters:
 - \$ref: "#/components/parameters/ProofIdPath"
 responses:
 "200":
 description: Proof
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/Proof" }
 "404":
 description: Not found
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/Error" }

/public/keys:
 get:
 tags: [Public]

summary: Get active Ed25519 public signing keys (for extension verification)

responses:

"200":

description: Keys

content:

application/json:

schema:

type: object

required: [keys]

properties:

keys:

type: array

items: { \$ref: "#/components/schemas/SigningKey" }

/billing/checkout:

post:

tags: [Billing]

summary: Create Stripe checkout session

security: [{ BearerAuth: [] }]

requestBody:

required: true

content:

application/json:

schema: { \$ref: "#/components/schemas/BillingCheckoutRequest" }

responses:

"200":

description: Checkout URL

content:

application/json:

schema: { \$ref: "#/components/schemas/BillingCheckoutResponse" }

/billing/webhook:

post:

tags: [Billing]

summary: Stripe webhook receiver

responses:

"200":

description: OK

/billing/status:

get:

tags: [Billing]

summary: Get current subscription status

security: [{ BearerAuth: [] }]

```
responses:
  "200":
    description: Status
    content:
      application/json:
        schema: { $ref: "#/components/schemas/BillingStatusResponse" }
```

/challenges:

```
post:
  tags: [Challenges]
  summary: Create a time-bound challenge (optional layer)
  security: [{ BearerAuth: [] }]
  requestBody:
    required: true
    content:
      application/json:
        schema: { $ref: "#/components/schemas/ChallengeCreateRequest" }
```

```
responses:
  "200":
    description: Created
    content:
      application/json:
        schema:
          type: object
          required: [id, public_url]
          properties:
            id: { type: string }
            public_url: { type: string }
```

```
"403":
  description: Not allowed (plan limit / channel not verified)
  content:
    application/json:
      schema: { $ref: "#/components/schemas/Error" }
```

/public/challenges/{challenge_id}:

```
get:
  tags: [Public, Challenges]
  summary: Get public challenge page data (viewer)
  parameters:
    - name: challenge_id
      in: path
      required: true
      schema: { type: string }
  responses:
```

"200":
 description: Challenge
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/ChallengePublic" }
"404":
 description: Not found
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/Error" }

/public/challenges/{challenge_id}/attempt:

post:
 tags: [Public, Challenges]
 summary: Submit viewer attempt
 parameters:
 - name: challenge_id
 in: path
 required: true
 schema: { type: string }
 requestBody:
 required: true
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/ChallengeAttemptRequest" }
 responses:
 "200":
 description: Attempt result
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/ChallengeAttemptResponse" }
 "429":
 description: Rate limited
 content:
 application/json:
 schema: { \$ref: "#/components/schemas/Error" }

/admin/reports:

get:
 tags: [Admin]
 summary: List reports (admin)
 security: [{ BearerAuth: [] }]
 parameters:
 - name: status

```

    in: query
    required: false
    schema: { type: string, enum: [open, reviewed, closed] }
  responses:
    "200":
      description: Reports
      content:
        application/json:
          schema:
            type: object
            required: [items]
            properties:
              items:
                type: array
                items:
                  type: object
                  additionalProperties: true
    "403":
      description: Forbidden
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Error" }

security: []

```

If I have to pick **one** for antiai.me (signing video proofs that will be verified a lot, including in a browser extension):

Pick: Ed25519 (EdDSA).

Why Ed25519 is the better default

- **Harder to misuse:** ECDSA (ES256) is notoriously easy to implement *incorrectly* if nonce generation is imperfect. Ed25519 is designed to avoid a lot of those foot-guns.
- **Fast + small:** Very fast verification and **small signatures** (64 bytes), great when your extension will verify constantly.
- **More “boring secure” in practice:** Ed25519 is widely used and well-studied; for typical product teams it’s often the safer operational choice.

When ES256 is the better pick

Choose **ES256** if you *must* align with ecosystems that strongly prefer NIST P-256 / JOSE defaults (some enterprise compliance environments, some existing KMS/HSM setups, or if you're deeply standardized around P-256 JWTs).

The honest security nuance

Pure cryptographic strength-wise, **both are strong** when implemented correctly.

But **real-world security** includes “how likely are we to screw it up”:

- **ES256 (ECDSA)**: secure *if* nonce handling + library usage is perfect.
- **Ed25519**: secure and generally **more robust against implementation mistakes**.

My recommendation for antiai.me

- Use **Ed25519** for proof tokens (`alg = EdDSA`).
- Add **key rotation** with `kid` and publish a public key endpoint for the extension:
 - `GET /public/keys` → list active public keys by `kid`
- If you later need enterprise compatibility, you can support **both** (EdDSA primary, ES256 optional).

If you tell me what language/stack you'll implement the signing in (Node, Python, Go), I'll recommend the best library patterns and how to format the token (raw payload+sig vs JWS) so the browser extension

Phase 1: YouTube + X (fastest to validate)

- YouTube proof issuance
- X profile token verification
- Extension: YouTube + X

Phase 2: Instagram + TikTok (harder but huge)

- Profile token + link-back verification
- Content URL proof
- Extension: Instagram + TikTok

Phase 3: Photo verification + “creator registry”

- Verified creator directory
- API for partners
- fraud reporting tools

Here's a clean, **minimal canonical payload** + a **browser-extension verification snippet (JS)** using Ed25519.

1) Canonical payload fields (minimal set)

Design goals

- Minimal fields, but enough to:
 1. bind proof to **exact YouTube video**
 2. bind to **exact YouTube channel**
 3. enforce **time validity**
 4. prevent replay/collision with a **nonce**
 5. support payload versioning

Canonical payload (v1)

Use exactly these fields:

```
{  
  "v": 1,  
  "iss": "antiai.me",
```

```
"kid": "k_2026_01",
"iat": 1739050000,
"exp": 1739654800,
"nonce": "3W9KxZqQv7cJ0cYQ0p1k0Q",
"youtube_video_id": "dQw4w9WgXcQ",
"youtube_channel_id": "UCxxxxxxxxxxxxxxxxxxxxxx"
}
```

Field definitions

- **v** (int): payload version. Start with **1**.
- **iss** (string): constant issuer, exactly **"antiai.me"**.
- **kid** (string): key id used to sign (must match **/public/keys** list).
- **iat** (int): issued-at as **Unix seconds** (UTC).
- **exp** (int): expiry as **Unix seconds** (UTC).
- **nonce** (string): **base64url** random 16 bytes (or 24+ chars) to ensure uniqueness.
- **youtube_video_id** (string): YouTube video id.
- **youtube_channel_id** (string): YouTube channel id.

That's it. Don't include names/handles/titles in the signed payload (those can change and don't affect authenticity).

2) Canonical serialization rule (must match server + extension)

Canonical JSON bytes

You need deterministic bytes to sign and verify.

Rule (simple and effective):

- Canonical JSON is built by:
 1. using UTF-8
 2. no whitespace
 3. keys sorted lexicographically
 4. strings escaped in standard JSON format

This is basically what libraries like **json-canonicalize (JCS)** do.

Recommendation: use JCS (JSON Canonicalization Scheme) to avoid subtle bugs.

So:

- Server does: `canonical = canonicalize(payloadObject) → bytes → base64url → sign`
- Extension does: `base64url decode → bytes → verify signature`

Important: **the extension does NOT need to canonicalize** if you return `payload_b64` from API. It verifies the bytes of `payload_b64` directly. That's the safest.

3) Reference verification snippet (Browser Extension JS)

Uses [@noble/ed25519](#) (reliable, modern)

This snippet:

1. fetches the proof for a YouTube video
2. fetches public keys
3. verifies Ed25519 signature
4. validates exp/iat + issuer + channel/video binding

Assumption: your API returns:

- `/public/proof?youtube_video_id=... → { alg, kid, payload_b64, signature_b64, expires_at }`
- `/public/keys → { keys: [{ kid, alg, public_key_b64 }] }`

Install dependency (for extension build)

```
npm i @noble/ed25519
```

Utility functions + verification

```
import * as ed25519 from "@noble/ed25519";
```

```
// --- base64url helpers ---
```

```
function b64urlToUint8Array(b64url) {
  // Convert base64url -> base64
  const b64 = b64url.replace(/-/g, "+").replace(/_/g, "/") + "===";
  const binary = atob(b64);
  const bytes = new Uint8Array(binary.length);
  for (let i = 0; i < binary.length; i++) bytes[i] = binary.charCodeAt(i);
  return bytes;
}
```

```
function b64ToUint8Array(b64) {
  // Standard base64 (not url)
```

```

const binary = atob(b64);
const bytes = new Uint8Array(binary.length);
for (let i = 0; i < binary.length; i++) bytes[i] = binary.charCodeAt(i);
return bytes;
}

function decodeUtf8(bytes) {
  return new TextDecoder().decode(bytes);
}

function safeJsonParse(str) {
  try { return JSON.parse(str); } catch { return null; }
}

// --- Fetch + cache keys (in-memory; you can persist via chrome.storage if desired) ---
let keyCache = null;
let keyCacheAt = 0;
const KEY_CACHE_TTL_MS = 6 * 60 * 60 * 1000; // 6 hours

async function getSigningKeys() {
  const now = Date.now();
  if (keyCache && (now - keyCacheAt) < KEY_CACHE_TTL_MS) return keyCache;

  const res = await fetch("https://api.antiai.me/public/keys", { method: "GET" });
  if (!res.ok) throw new Error("Failed to fetch keys");
  const data = await res.json();

  // Build map: kid -> publicKeyBytes
  const map = new Map();
  for (const k of data.keys || []) {
    if (k.alg !== "Ed25519") continue;
    // public_key_b64 is standard base64 of raw 32 bytes
    map.set(k.kid, b64ToUint8Array(k.public_key_b64));
  }

  keyCache = map;
  keyCacheAt = now;
  return map;
}

// --- Core verification function ---
export async function verifyVideoProof({ youtubeVideoid, expectedYoutubeChannelId = null }) {
  // 1) fetch proof token
  const proofRes = await fetch(

```

```

`https://api.antiai.me/public/proof?youtube_video_id=${encodeURIComponent(youtubeVideoId)}
`,
  { method: "GET" }
);

if (proofRes.status === 404) {
  return { ok: false, status: "unverified", reason: "no_active_proof" };
}
if (!proofRes.ok) {
  return { ok: false, status: "error", reason: "proof_fetch_failed" };
}

const proof = await proofRes.json();
if (proof.alg !== "Ed25519") {
  return { ok: false, status: "error", reason: "unsupported_alg" };
}

const payloadBytes = b64urlToUint8Array(proof.payload_b64);
const sigBytes = b64urlToUint8Array(proof.signature_b64);

// 2) get public key by kid
const keys = await getSigningKeys();
const pubKey = keys.get(proof.kid);
if (!pubKey) {
  return { ok: false, status: "error", reason: "unknown_kid" };
}

// 3) verify Ed25519 signature
const signatureOk = await ed25519.verify(sigBytes, payloadBytes, pubKey);
if (!signatureOk) {
  return { ok: false, status: "error", reason: "bad_signature" };
}

// 4) parse payload (payload bytes contain canonical JSON)
const payloadStr = decodeUtf8(payloadBytes);
const payload = safeJsonParse(payloadStr);
if (!payload) {
  return { ok: false, status: "error", reason: "invalid_payload_json" };
}

// 5) validate minimal fields
// Required exact values
if (payload.v !== 1) return { ok: false, status: "error", reason: "bad_version" };

```

```

if (payload.iss !== "antiai.me") return { ok: false, status: "error", reason: "bad_issuer" };
if (payload.kid !== proof.kid) return { ok: false, status: "error", reason: "kid_mismatch" };

// Bindings
if (payload.youtube_video_id !== youtubeVideoid) {
  return { ok: false, status: "error", reason: "video_id_mismatch" };
}
if (expectedYoutubeChannelId && payload.youtube_channel_id !==
expectedYoutubeChannelId) {
  return { ok: false, status: "error", reason: "channel_id_mismatch" };
}

// Time validity (iat/exp as unix seconds)
const nowSec = Math.floor(Date.now() / 1000);
const iat = Number(payload.iat);
const exp = Number(payload.exp);

// Allow a small clock skew (e.g., 5 minutes)
const SKEW_SEC = 5 * 60;

if (!Number.isFinite(iat) || !Number.isFinite(exp)) {
  return { ok: false, status: "error", reason: "bad_time_fields" };
}
if (iat > nowSec + SKEW_SEC) {
  return { ok: false, status: "error", reason: "iat_in_future" };
}
if (exp <= nowSec - SKEW_SEC) {
  return { ok: false, status: "expired", reason: "proof_expired" };
}
if (exp <= iat) {
  return { ok: false, status: "error", reason: "exp_before_iat" };
}

// Nonce basic sanity (presence)
if (typeof payload.nonce !== "string" || payload.nonce.length < 16) {
  return { ok: false, status: "error", reason: "bad_nonce" };
}

// All checks passed
return {
  ok: true,
  status: "verified",
  payload, // optional: return for UI details
};

```

}

4) How the extension uses this

On a YouTube watch page:

1. Extract `youtube_video_id` from URL (`v=` query param).
2. Call `verifyVideoProof({ youtubeVideoId })`.
3. Render badge:
 - `verified` →  Verified
 - `unverified` →  Unverified
 - `expired` →  Expired
 - `error` →  Error / Suspicious

5) Server-side signing (rules the backend must follow)

When issuing or reissuing:

- Build payload object with the exact minimal fields above.
- Create canonical JSON bytes via JCS (recommended).
- `payload_b64 = base64url(payload_bytes)`
- `signature = Ed25519.sign(payload_bytes, privateKey)`
- `signature_b64 = base64url(signature)`

Return `kid` used.

a **server-side Node.js signing function** using `@noble/ed25519`

```
// signing.js

// Server-side Ed25519 signing for antiai.me using @noble/ed25519

// - Builds the minimal canonical payload

// - Canonicalizes JSON using JCS (json-canonicalize)

// - Produces payload_b64 (base64url of canonical JSON bytes)

// - Produces signature_b64 (base64url of 64-byte Ed25519 signature)
```

```

import * as ed25519 from "@noble/ed25519";

import { canonicalize } from "json-canonicalize"; // npm i json-canonicalize

// ----- base64url helpers -----

function toBase64Url(bytes) {
  const b64 = Buffer.from(bytes).toString("base64");
  return b64.replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/g, "");
}

function fromBase64(b64) {
  return new Uint8Array(Buffer.from(b64, "base64"));
}

function nowUnixSec() {
  return Math.floor(Date.now() / 1000);
}

function randomNonceB64Url(byteLen = 16) {
  // Node 18+:
  const { randomBytes } = await import("crypto");
  const buf = randomBytes(byteLen);
  return toBase64Url(buf);
}

// ----- strict payload builder (minimal set) -----

export async function buildCanonicalPayload({

```

```
kid,
youtubeVideoid,
youtubeChannelId,
expUnixSec,
}) {
  if (!kid) throw new Error("kid is required");
  if (!youtubeVideoid) throw new Error("youtubeVideoid is required");
  if (!youtubeChannelId) throw new Error("youtubeChannelId is required");

  const iat = nowUnixSec();
  const exp = Number(expUnixSec);
  if (!Number.isFinite(exp)) throw new Error("expUnixSec must be a number");
  if (exp <= iat) throw new Error("exp must be > iat");

  // Minimal canonical payload fields
  const payload = {
    v: 1,
    iss: "antiai.me",
    kid,
    iat,
    exp,
    nonce: await randomNonceB64Url(16),
    youtube_video_id: youtubeVideoid,
    youtube_channel_id: youtubeChannelId,
  };

  // Canonical JSON string per JCS
```

```

const canonicalJson = canonicalize(payload); // stable key ordering, minimal formatting
const payloadBytes = new TextEncoder().encode(canonicalJson);

return { payload, canonicalJson, payloadBytes };
}

```

```

// ----- signing function -----

```

```

export async function signProofEd25519({
  kid,
  youtubeVideoid,
  youtubeChannelId,
  expiresAt,    // Date | number (ms) | unix seconds (if unixExp=true)
  unixExp = false, // if true, expiresAt is unix seconds
  privateKeyB64, // base64 of raw 32-byte Ed25519 private key seed OR 64-byte
secret key
}) {

```

```

  if (!privateKeyB64) throw new Error("privateKeyB64 is required");

```

```

  // Normalize exp to unix seconds

```

```

  let expUnixSec;

```

```

  if (unixExp) {

```

```

    expUnixSec = Number(expiresAt);

```

```

  } else if (expiresAt instanceof Date) {

```

```

    expUnixSec = Math.floor(expiresAt.getTime() / 1000);

```

```

  } else if (typeof expiresAt === "number") {

```

```

    // assume ms

```

```

    expUnixSec = Math.floor(expiresAt / 1000);

```

```

  } else {

```



```
    throw new Error("expiresAt must be Date or number");
  }
```

```
const { payload, canonicalJson, payloadBytes } = await buildCanonicalPayload({
  kid,
  youtubeVideoid,
  youtubeChannelId,
  expUnixSec,
});
```

```
// Decode private key material
```

```
const privBytes = fromBase64(privateKeyB64);
```

```
// @noble/ed25519 expects 32-byte private key (seed).
```

```
// If you have a 64-byte secret key, slice first 32 bytes.
```

```
const priv32 = privBytes.length === 32 ? privBytes : privBytes.slice(0, 32);
```

```
// Sign canonical payload bytes
```

```
const sigBytes = await ed25519.sign(payloadBytes, priv32);
```

```
// Produce base64url outputs
```

```
const payload_b64 = toBase64Url(payloadBytes);
```

```
const signature_b64 = toBase64Url(sigBytes);
```

```
return {
  alg: "Ed25519",
  kid,
```

```

    // Stored in DB

    payload_json: payload,          // for inspection only
    payload_b64,                    // base64url(canonical JSON bytes)
    signature_b64,                  // base64url(signature 64 bytes)
    issued_at_unix: payload.iat,
    expires_at_unix: payload.exp,
    canonical_json: canonicalJson, // optional for debugging
  };
}

// ----- verification helper (server-side sanity check) -----
export async function verifyProofEd25519({
  payload_b64,
  signature_b64,
  publicKeyB64, // base64 of raw 32-byte Ed25519 public key
}) {
  const payloadBytes = base64UrlToBytes(payload_b64);
  const sigBytes = base64UrlToBytes(signature_b64);
  const pubBytes = fromBase64(publicKeyB64);

  const ok = await ed25519.verify(sigBytes, payloadBytes, pubBytes);
  return ok;
}

function base64UrlToBytes(b64url) {
  const b64 = b64url.replace(/-/g, "+").replace(/_/g, "/") + "===";
  const bytes = Buffer.from(b64, "base64");
  return new Uint8Array(bytes);
}

```

```
}
```

Install dependencies

```
npm i @noble/ed25519 json-canonicalize
```

Notes (important)

- **Private key format:** `@noble/ed25519` expects a **32-byte private key seed**.
If you store a 64-byte secret key, the snippet slices the first 32 bytes.
- **Canonicalization:** `json-canonicalize` (JCS) ensures stable bytes for signing.
- **DB storage:** store `payload_json`, `payload_b64`, `signature_b64`, `kid`, `expires_at`.

If you tell me where you plan to store keys (env vars vs AWS KMS vs GCP KMS), I can adjust this into a production-grade key-loader (including rotation and `kid` selection logic).

----- tiny **content-script snippet** that injects the badge into YouTube UI cleanly.

```
// content-script.js
```

```
// Tiny YouTube badge injector for antiai.me verification
```

```
// Assumes you have verifyVideoProof({ youtubeVideoId }) available (import or bundle it).
```

```
// Works on YouTube SPA navigation by watching URL changes + DOM readiness.
```

```
// ----- CONFIG -----
```

```
const BADGE_ID = "antiai-verify-badge";
```

```
const API_TIMEOUT_MS = 8000;
```

```
// ----- HELPERS -----
```

```
function getYouTubeVideoId() {
```

```
  const url = new URL(location.href);
```

```
  // Standard watch URL: https://www.youtube.com/watch?v=VIDEOID
```

```
  const v = url.searchParams.get("v");
```

```
  if (v) return v;
```

```

// Shorts: https://www.youtube.com/shorts/VIDEOID
const shortsMatch = url.pathname.match(/\/shorts\/([a-zA-Z0-9_-]{6,})/);
if (shortsMatch) return shortsMatch[1];

return null;
}

function waitForElement(selector, { timeoutMs = 8000 } = {}) {
  return new Promise((resolve) => {
    const existing = document.querySelector(selector);
    if (existing) return resolve(existing);

    const obs = new MutationObserver(() => {
      const el = document.querySelector(selector);
      if (el) {
        obs.disconnect();
        resolve(el);
      }
    });
    obs.observe(document.documentElement, { childList: true, subtree: true });

    setTimeout(() => {
      obs.disconnect();
      resolve(null);
    }, timeoutMs);
  });
}

function ensureBadge() {
  let badge = document.getElementById(BADGE_ID);
  if (!badge) {
    badge = document.createElement("div");
    badge.id = BADGE_ID;
    badge.style.cssText = `
      display: inline-flex;
      align-items: center;
      gap: 8px;
      padding: 6px 10px;
      border-radius: 999px;
      font-size: 12px;
      line-height: 1;
      border: 1px solid rgba(255,255,255,0.15);
      background: rgba(0,0,0,0.35);
      color: #fff;
    `;
  }
}

```

```
    cursor: default;
    user-select: none;
    margin-left: 8px;
    backdrop-filter: blur(8px);
    -webkit-backdrop-filter: blur(8px);
`;
```

```
const dot = document.createElement("span");
dot.dataset.dot = "1";
dot.style.cssText = `
    width: 8px;
    height: 8px;
    border-radius: 50%;
    background: rgba(255,255,255,0.6);
    display: inline-block;
    flex: 0 0 auto;
`;
```

```
const text = document.createElement("span");
text.dataset.text = "1";
text.textContent = "Checking...";
```

```
    badge.append(dot, text);
}
return badge;
}
```

```
function setBadgeState(badge, state) {
    const dot = badge.querySelector("[data-dot='1']");
    const text = badge.querySelector("[data-text='1']");

    const states = {
        checking: { dot: "rgba(255,255,255,0.6)", text: "Checking..." },
        verified: { dot: "#22c55e", text: "Verified by antiai.me" },
        unverified: { dot: "#f59e0b", text: "Unverified" },
        expired: { dot: "#f59e0b", text: "Proof expired" },
        error: { dot: "#ef4444", text: "Verification error" },
    };

    const s = states[state] || states.error;
    dot.style.background = s.dot;
    text.textContent = s.text;
    badge.title = badge.title || "antiai.me: authenticity signal for creators";
}
```

```

// YouTube watch page title area
// This selector is relatively stable: title + metadata line sits under #title
async function findBadgeAnchor() {
  // Prefer the title row container
  // YouTube changes often; we use a fallback chain.
  const selectors = [
    "ytd-watch-metadata #title h1",    // classic watch
    "ytd-watch-metadata #title",      // fallback
    "ytd-watch-metadata",             // fallback
    "ytd-reel-video-renderer h2",     // shorts / reels-ish
  ];

  for (const sel of selectors) {
    const el = await waitForElement(sel, { timeoutMs: 3000 });
    if (el) return el;
  }
  return null;
}

function abortableTimeout(ms) {
  const controller = new AbortController();
  const t = setTimeout(() => controller.abort(), ms);
  return { controller, cancel: () => clearTimeout(t) };
}

// ----- MAIN LOGIC -----
let lastVideoid = null;

async function injectOrUpdateBadge() {
  const videoid = getYouTubeVideoid();
  if (!videoid) return;

  if (videoid === lastVideoid && document.getElementById(BADGE_ID)) return;
  lastVideoid = videoid;

  const anchor = await findBadgeAnchor();
  if (!anchor) return;

  // Insert badge next to title (clean + visible)
  const badge = ensureBadge();
  setBadgeState(badge, "checking");

  // Ensure it's placed

```

```

// If anchor is h1, insert after it; otherwise append into anchor.
if (!badge.isConnected) {
  if (anchor.tagName === "H1") {
    anchor.insertAdjacentElement("afterend", badge);
  } else {
    anchor.appendChild(badge);
  }
}

// Run verification (with timeout safety)
try {
  const { controller, cancel } = abortableTimeout(API_TIMEOUT_MS);

  // If your verifyVideoProof doesn't accept abort signals, it's fine; this just prevents hangs in
  fetch you control.
  // You can wire controller.signal into your fetch calls if desired.
  const result = await verifyVideoProof({ youtubeVideoid: videoid });
  cancel();

  if (result.ok && result.status === "verified") setBadgeState(badge, "verified");
  else if (result.status === "unverified") setBadgeState(badge, "unverified");
  else if (result.status === "expired") setBadgeState(badge, "expired");
  else setBadgeState(badge, "error");
} catch (e) {
  setBadgeState(badge, "error");
}

// YouTube is a SPA. Detect navigation changes.
function startYouTubeSpaWatcher() {
  let prev = location.href;

  // 1) Polling approach (simple + reliable)
  setInterval(() => {
    if (location.href !== prev) {
      prev = location.href;
      // Remove old badge to avoid weird placements
      const old = document.getElementById(BADGE_ID);
      if (old) old.remove();
      injectOrUpdateBadge();
    }
  }, 700);

  // 2) Also run on first load + when DOM changes (for late-loaded metadata)

```

```
injectOrUpdateBadge();
const obs = new MutationObserver(() => {
  // If badge missing but we have a video id, try again (lightweight)
  if (getYouTubeVideoId() && !document.getElementById(BADGE_ID)) {
    injectOrUpdateBadge();
  }
});
obs.observe(document.documentElement, { childList: true, subtree: true });
}

// Boot
startYouTubeSpaWatcher();
```

Below is the **final, locked structure** for both **Creator** and **Admin** dashboards, with the **MVP scope clearly defined** (what's in v1 vs later). This is what you can hand to designers + developers.

Creator Dashboard

Creator navigation (final)

Sidebar

1. **Dashboard**
2. **Channels**
3. **Videos**
4. **Billing**
5. **Settings**

(Future)

- Proofs (optional; can stay inside Videos)
- Analytics (Pro+)

- Challenges (Pro+)
 - Team (Elite)
 - API (Elite)
-

Creator MVP screens and flow

1) Dashboard (Home)

Purpose: show status + guide next action

MVP widgets

- Verification status (YouTube verified or not)
- Plan + usage meter (e.g., **3/10 videos used**)
- Quick action buttons:
 - **Verify YouTube**
 - **Add Video**
 - **Upgrade**

Empty states (must-have)

- If YouTube not verified → big “Verify YouTube to start”
 - If verified but no videos → “Add your first video”
-

2) Channels

Purpose: verify identity (starting with YouTube)

MVP

- YouTube channel connect + verification (OAuth preferred)

- Show channel card:
 - channel name, handle, channel id
 - status: pending/verified/revoked
 - verified date

(Future)

- Add social accounts: X, Instagram, TikTok, LinkedIn, Website
 - Social proof badges on creator page
-

3) Videos (Core)

Purpose: main workflow for creators

MVP

- Paste YouTube URL → system checks ownership → registers video
- “Generate Proof” (or auto-generate after verification)
- Video list table:
 - title, youtube_video_id, status, proof status, expires at
- Video detail drawer/page:
 - verification result
 - active proof
 - buttons:
 - Copy verification link
 - View public creator page
 - Reissue proof (Pro+ or allowed for security/expiry)

Usage enforcement

- Free: 10/month
- Pro: 100/month
- Elite: unlimited

(Future)

- Bulk import videos
 - Auto-sync latest uploads
 - Proof history view (superseded chain)
 - Optional challenges per video
-

4) Billing

MVP

- Current plan
 - Usage meter
 - Upgrade / downgrade buttons
 - Payment method + invoices (Stripe portal link)
-

5) Settings

MVP

- Profile basics
- Change password
- Delete account

- Notification preferences (basic)
-

Creator MVP flow (final)

1. Signup/Login
 2. Dashboard (locked)
 3. Verify YouTube
 4. Add video
 5. Proof generated
 6. Share creator page: antiai.me/creators/{handle}
 7. Upgrade when limits are reached
-

Admin Dashboard

Admin navigation (final)

Sidebar

1. Overview
2. Creators
3. Channels
4. Videos
5. Proofs
6. Reports

7. **Signing Keys**

8. **Billing**

9. **System Logs**

(Future)

- Roles & permissions
- Advanced analytics
- Support inbox / ticketing

Admin MVP screens and flow

1) Overview

MVP widgets

- Total creators
- Verified channels
- Proofs issued (24h / 7d)
- Active proofs
- Reports open
- Revenue snapshot (MRR)

MVP alerts

- Spike in verification checks
- Stripe webhook errors
- Too many reports for one creator/channel

- Key retirement reminder
-

2) Creators

MVP

- Search by email / user id
 - Table with:
 - plan, status, channels count, created date
 - Creator detail page:
 - channels list
 - videos count
 - proofs count
 - billing status
 - Actions:
 - suspend creator
 - grant Pro temporarily
 - reset usage limits (manual support)
-

3) Channels

MVP

- Table:
 - channel id, handle/name, owner email, status
- Channel detail:

- verification method + time
 - related videos and proofs
 - Actions:
 - revoke channel
 - mark suspicious
 - internal notes
-

4) Videos

MVP

- Search by youtube_video_id
 - View which channel/user registered it
 - Links to proof + creator page
 - Flags: suspicious / normal
-

5) Proofs (Core)

MVP

- Filter:
 - active / expired / revoked / superseded
- Proof detail:
 - payload summary
 - kid
 - iat/exp

- chain (supersedes pointer)
 - Actions:
 - revoke (security)
 - force reissue (admin override)
 - view transparency log events
-

6) Reports

MVP

- Report queue:
 - open / reviewed / closed
 - Report detail:
 - reason + evidence links
 - affected channel/video/proof
 - Actions:
 - close
 - revoke proof
 - suspend creator
 - mark as false report
-

7) Signing Keys (Cryptographic control)

MVP

- Keys list:

- active/retired
- created date
- proofs signed count
- Actions:
 - create new key (if you want admin-generated)
 - set active key
 - retire key

(Future)

- scheduled rotation
 - staged rollout
 - incident mode (rapid key disable + forced reissue)
-

8) Billing

MVP

- Subscription list (plan/status)
 - Search by email
 - Link to Stripe customer/subscription
-

9) System Logs

MVP

- Recent events from `audit_logs` / `transparency_log`
- Filters by action/event_type

- Export CSV (optional)
-

Admin MVP operational flow (final)

1. Monitor system health (Overview)
 2. Handle support (Creators/Billing)
 3. Handle abuse (Reports → Channels/Proofs actions)
 4. Maintain crypto trust layer (Signing Keys → rotation/retirement)
-

MVP summary checklist (final)

Creator MVP (v1 includes)

- ✓ Signup/Login
- ✓ Verify YouTube channel
- ✓ Add video (paste URL)
- ✓ Issue proof (Ed25519)
- ✓ View/share creator page
- ✓ Plan limits (Free 10 / Pro 100 / Elite unlimited)
- ✓ Billing upgrade flow

Admin MVP (v1 includes)

- ✓ Overview metrics + alerts
- ✓ Manage creators/channels/videos/proofs
- ✓ Reports queue + moderation actions
- ✓ Signing keys management
- ✓ Billing visibility + Stripe linking
- ✓ Logs (audit + transparency)

AntiAi.me Minimum Viable Product (MVP) Summary Creator Features (v1)

The Creator MVP focuses on providing users with the core functionality to verify content ownership and issue proofs.

- **Account Management:** Signup and Login capabilities.
- **Channel Verification:** Ability to verify a user's YouTube channel.
- **Video Integration:** Simple process for adding a video by pasting its URL.
- **Proof Issuance:** Generation of secure proofs (Ed25519) for content ownership.
- **Public Presence:** Option to view and share the creator's dedicated page.
- **Usage Tiers:** Defined limits based on subscription plans:
 - Free: 10 proofs
 - Pro: 100 proofs
 - Elite: Unlimited proofs
- **Monetization:** Functionality for users to upgrade their billing plan.

Admin Features (v1)

The Admin MVP is designed to provide comprehensive tools for platform oversight, moderation, and technical management.

- **Platform Monitoring:** Access to key overview metrics and immediate alerts.
- **Content Management:** Tools to manage creators, channels, videos, and issued proofs.
- **Moderation & Reporting:** Visibility into the reports queue and execution of moderation actions.
- **Security Management:** Control over signing keys.
- **Financial Oversight:** Billing visibility and integration with Stripe.
- **Transparency & Audit:** Comprehensive logging for both audit and transparency purposes.